SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0714

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards *(BL2,BL6)*

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE
802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

ANSWERS

Aim : To write a Python function named `validate_segment(length_m)` to verify whether a UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters.

Python program

```
def validate_segment(length_m):
    max_length = 100

    if length_m <= max_length:
        return True, "Valid cable length"
    else:
        return False, "Error: Cable length exceeds the maximum limit of 100 meters"
# Test cases
test_cases = [50, 100, 120]
for length in test_cases:
    result, message = validate_segment(length)
    print("Cable length:", length, "m")
    print("Result:", result)
    print("Message:", message)
    print()
```

Python program implementation

```python
def validate_segment(length_m):
    max_length = 100

    if length_m <= max_length:
        return True, "Valid cable length"
    else:
        return False, "Error: Cable length exceeds the maximum limit of 100 meters"


# Test cases
test_cases = [50, 100, 120]

for length in test_cases:
    result, message = validate_segment(length)
    print("Cable length:", length, "m")
    print("Result:", result)
    print("Message:", message)
    print()
```

Output

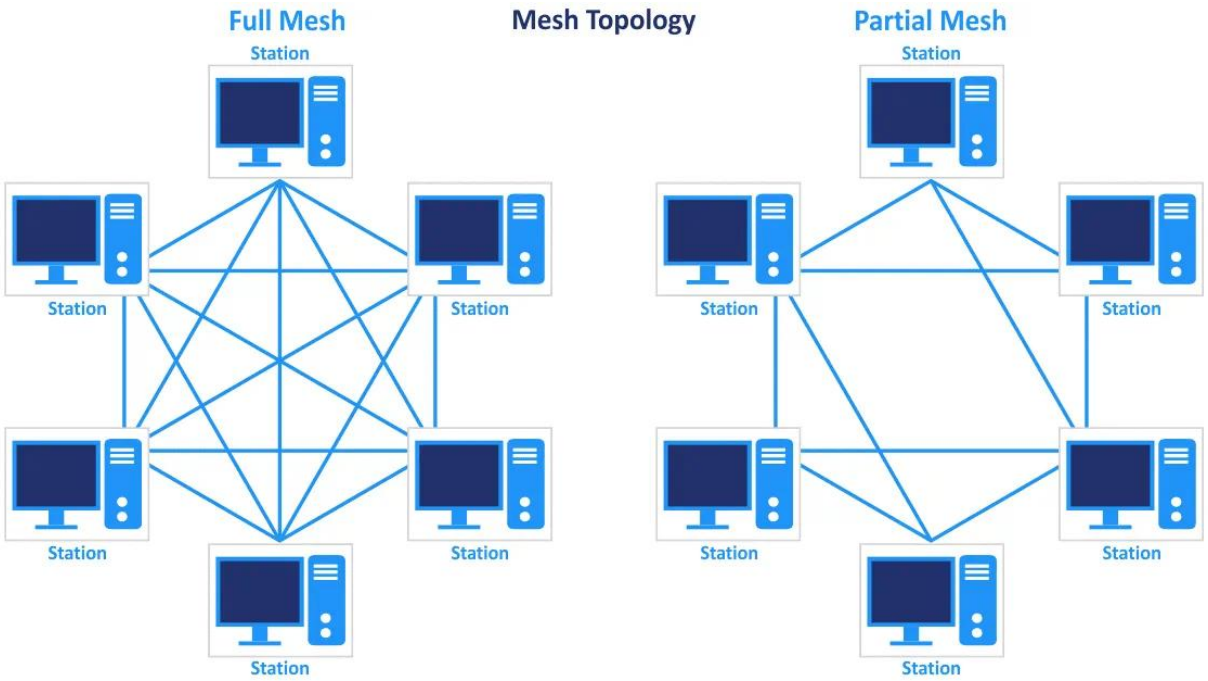
```
ERROR!
Cable length: 50 m
Result: True
Message: Valid cable length

Cable length: 100 m
Result: True
Message: Valid cable length

Cable length: 120 m
Result: False
Message: Error: Cable length exceeds the maximum limit of 100 meters


=== Code Execution Successful ===
```

Explanation

Interpretation of Results

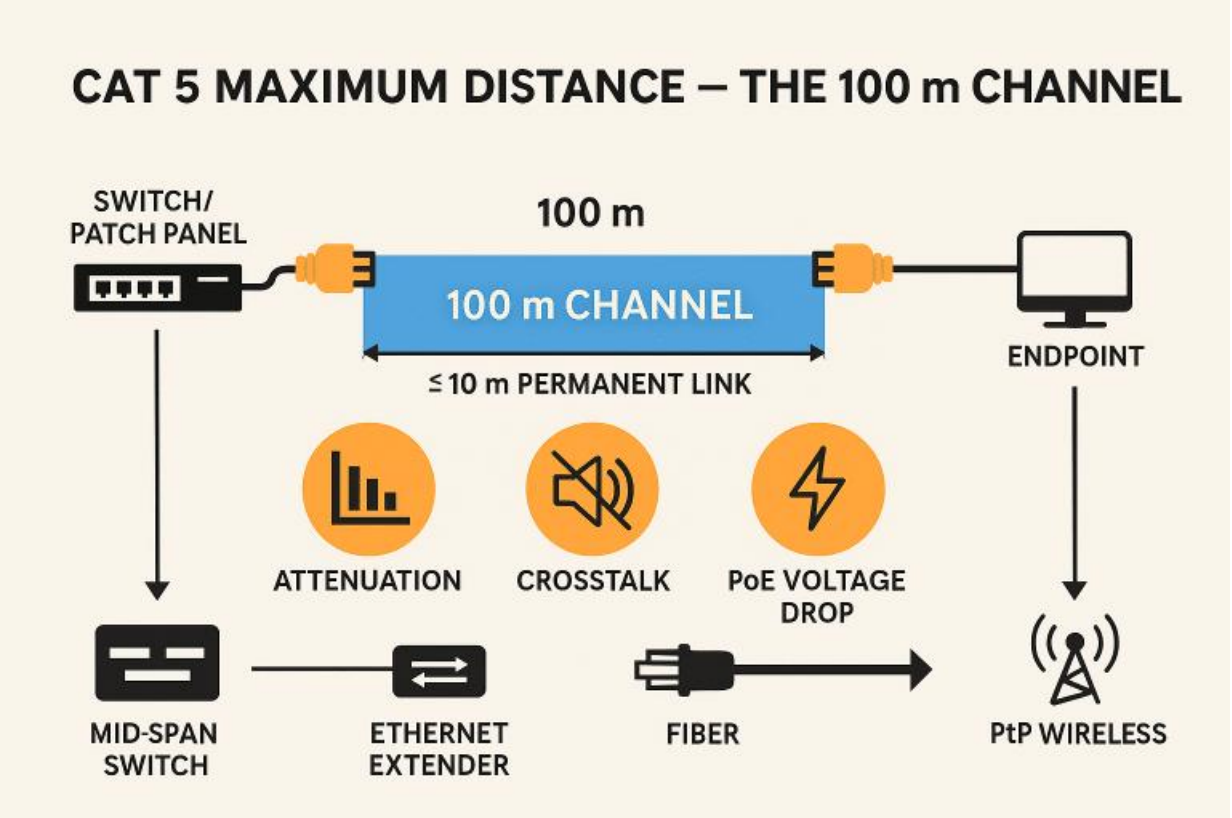
1. 50 meters: The cable length is below the 100-meter limit, so it is valid.
2. 100 meters: The cable length is exactly equal to the maximum permissible limit, so it is valid.
3. 120 meters: The cable length exceeds the 100-meter limit, so it is invalid and an error message is displayed.

Visual Representation :



Explanation :

- **Full Mesh:** Every station/computer is directly connected to every other station. This provides high reliability because multiple communication paths are available.
- **Partial Mesh:** Only some stations are directly connected to each other. It uses fewer connections than a full mesh while still providing alternative communication paths.



- The diagram represents the maximum 100-meter Ethernet channel length for UTP cable.
- It shows the connection between a switch/patch panel and an endpoint.
- The 100-meter limit is important for maintaining reliable Ethernet communication.
- This concept is implemented in the Python program by accepting cable lengths up to 100 meters and rejecting lengths that exceed the limit.

Conclusion

The Python program successfully validates UTP cable segment lengths according to the given IEEE 802.3 requirement. It correctly accepts cable lengths up to 100 meters and rejects lengths exceeding 100 meters.

2. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Aim :

To develop a client-server application using Python's `socket` library to simulate communication between two hosts connected through a central switch in a star topology. The server receives frames, learns MAC addresses, forwards frames to the appropriate port, and displays the updated MAC address table.

Python code :

```
import socket
import threading

HOST = "127.0.0.1"
PORT = 5000

# MAC Address Table: MAC Address -> Port
mac_table = {}

# Connected clients: Port -> Socket
clients = {}


def display_mac_table():
    print("\n----- MAC ADDRESS TABLE -----")
    print("MAC Address            Port")

    for mac, port in mac_table.items():
        print(f"{mac:<24} {port}")

    print("-----------------------------")


def handle_client(conn, port):

    # Receive client's MAC address
    mac = conn.recv(1024).decode()

    # Learn MAC address
    mac_table[mac] = port
    clients[port] = conn

    print(f"\nHost connected: {mac} -> Port {port}")

    display_mac_table()
```

```
    while True:

        data = conn.recv(1024).decode()

        if not data:
            break

        # Frame format: destination MAC | message
        destination_mac, message = data.split("|", 1)

        print("\nFrame Received")
        print("Source MAC      :", mac)
        print("Destination MAC :", destination_mac)
        print("Message         :", message)

        # Learn source MAC
        mac_table[mac] = port

        # Check destination MAC
        if destination_mac in mac_table:

            destination_port = mac_table[destination_mac]

            # Forward frame to correct port
            clients[destination_port].send(
                f"{mac}|{message}".encode()
            )

            print(
                f"Frame forwarded from Port {port} "
                f"to Port {destination_port}"
            )

            print("\nUpdated MAC Address Table:")
            display_mac_table()

        else:
            print("Destination MAC unknown.")
            print("Frame is broadcast to other ports.")

            for p, client in clients.items():
                if p != port:
                    client.send(
                        f"{mac}|{message}".encode()
                    )

            display_mac_table()


# Create server socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind((HOST, PORT))
server.listen(5)
```

```
print("Central Switch Started")
print("Waiting for hosts...")

port = 1

while True:

    conn, address = server.accept()

    thread = threading.Thread(
        target=handle_client,
        args=(conn, port)
    )

    thread.start()

    port += 1
```

Program Implementation:

```
int("MAC Address                Port")

or mac, port in mac_table.items():
    print(f"{mac:<24} {port}")

rint("----------------------------")


andle_client(conn, port):

 Receive client's MAC address
ac = conn.recv(1024).decode()

 Learn MAC address
ac_table[mac] = port
lients[port] = conn

rint(f"\nHost connected: {mac} -> Port {port}")

isplay_mac_table()

hile True:
    data = conn.recv(1024).decode()

    if not data:
        break
```

```
destination_mac, message = data.split("|", 1)

print("\nFrame Received")
print("Source MAC      :", mac)
print("Destination MAC :", destination_mac)
print("Message         :", message)

# Learn source MAC
mac_table[mac] = port

# Check destination MAC
if destination_mac in mac_table:

    destination_port = mac_table[destination_mac]

    # Forward frame to correct port
    clients[destination_port].send(
        f"{mac}|{message}".encode()
    )

    print(
        f"Frame forwarded from Port {port} "
        f"to Port {destination_port}"
    )
```

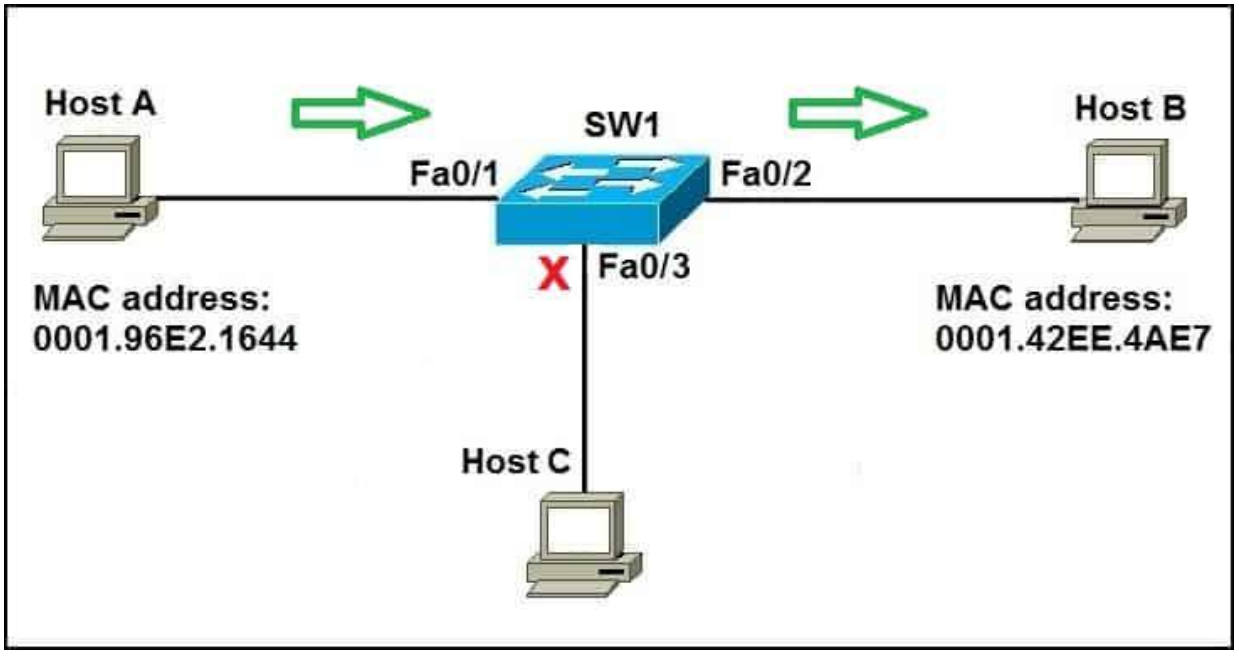
Output :

```
Central Switch Started
Waiting for hosts...
Host connected: AA:AA:AA:AA:AA:01 -> Port 1
----- MAC ADDRESS TABLE -----
MAC Address               Port
------------------------------
AA:AA:AA:AA:AA:01         1
AA:AA:AA:AA:AA:01         1
------------------------------
Host connected: BB:BB:BB:BB:BB:02 -> Port 2
----- MAC ADDRESS TABLE -----
MAC Address               Port
------------------------------
AA:AA:AA:AA:AA:01         1
BB:BB:BB:BB:BB:02         2
------------------------------
Frame Received
Source MAC       : AA:AA:AA:AA:AA:01
Destination MAC  : BB:BB:BB:BB:BB:02
Message          : Hello Host B
Frame forwarded from Port 1 to Port 2
Updated MAC Address Table:
----- MAC ADDRESS TABLE -----
MAC Address               Port
------------------------------
AA:AA:AA:AA:AA:01         1
BB:BB:BB:BB:BB:02         2
------------------------------
```

Explanation :

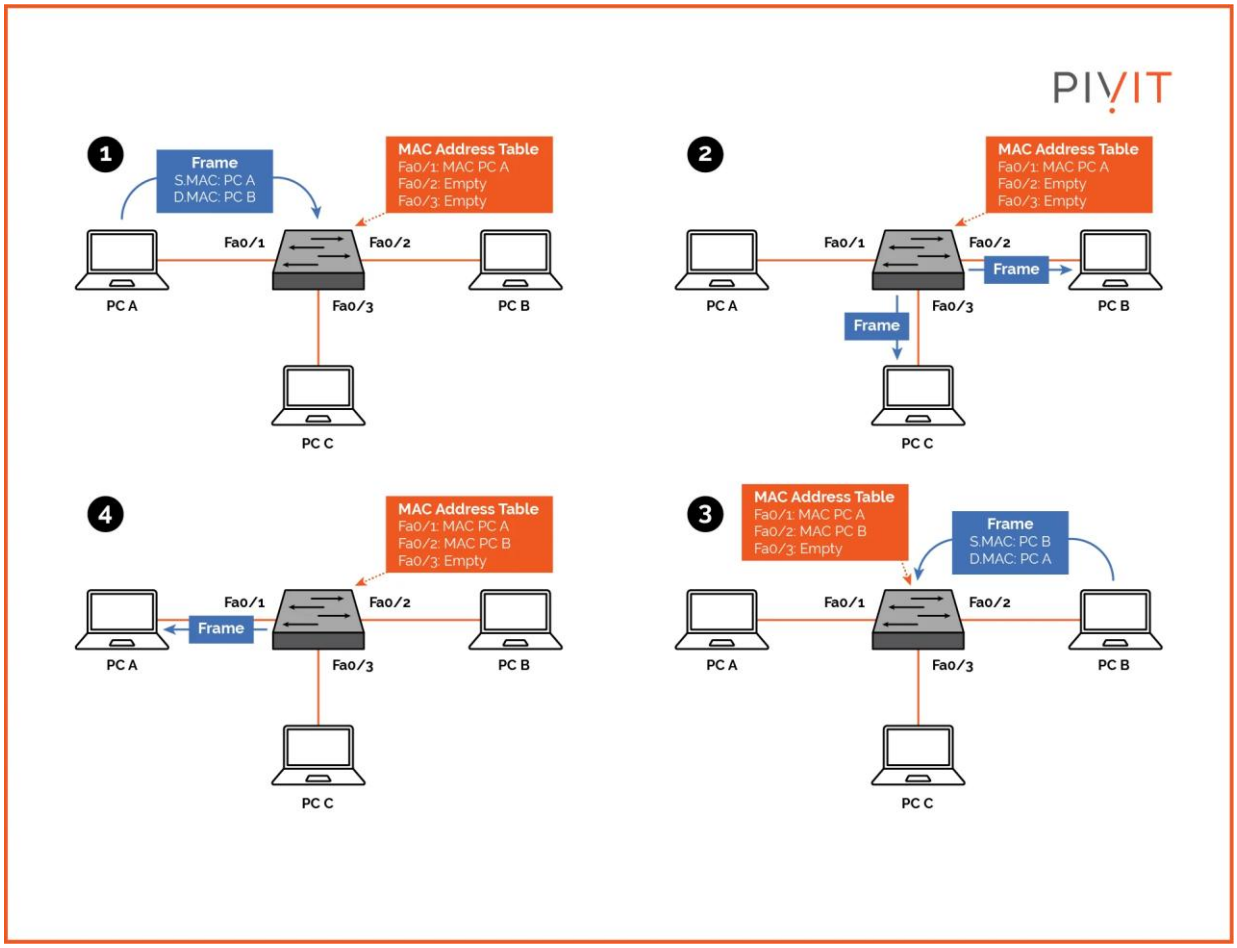
- The server acts as the central switch and assigns a port to each connected client.
- Each client provides its MAC address, which the server stores in the simulated MAC address table.
- When a client sends a frame, the server reads the source and destination MAC addresses.
- The server learns the source MAC address and checks the MAC table for the destination address.
- If the destination is known, the frame is forwarded to the corresponding port.
- After transmission, the updated MAC address table is displayed.

Visual Representation :



Explanation :

The image demonstrates how an Ethernet switch uses MAC addresses and port information to forward a frame only toward the correct destination. This is directly related to your Python program's MAC learning and frame forwarding.



Conclusion

The client-server application successfully simulated communication between two hosts through a central switch using Python sockets. The simulation demonstrated **MAC address learning, MAC table maintenance, and frame forwarding**, showing how Ethernet switches identify the destination MAC address and forward frames through the appropriate port.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done